

# 连铸切割的在线优化与最优性验证

## 摘要

针对连铸尾坯处理与异常通知下的在线切割问题，本文将一次切割的运输长度限制与离线回收的产品长度要求分开建模，以报废总长度为第一目标、合格产品偏离目标长度的平方和为第二目标，形成尾坯解析分配与事件驱动动态规划相结合的求解方法。

**针对问题一：**枚举合格产品数及整块报废件数，推导每种计数下的最大回收量，再由凸性确定等长分配。给出全部 12 种尾坯方案；其中 109.0 米和 93.4 米均可零损失回收，14.5 米可回收 9.7 米。严格执行尾坯也需满足 4.8 米运输下限时，13.7 米尾坯没有合格产品可回收。

**针对问题二：**按异常通知逐次更新已知报废区间，锁定已启动的切口，在 0.1 米网格上用字典序最短路重新规划。9.5 米目标下，给定 9 次异常序列的最终损失为 23.8 米，达到连续材料分割理论下界。

**针对问题三：**保持信息边界和求解方法，仅替换产品区间。8.5 米与 11.1 米目标下，最终损失分别为 16.0 米和 25.9 米，也达到相应下界。三组结果均给出原待执行方案、新方案和剩余计划损失。

独立以单元占用图构造两阶段最小费用流线性规划，12 种尾坯损失及 27 次事件的两级目标均与主算法一致。实测单次规划最多 0.042 秒；网格加密不改变三组损失，但会改善长度偏差。改变初始切口相位可改变最小损失，因此最优性结论明确限定初始状态、零切缝和所声明的二级偏好，不外推为未知未来异常下的普遍最优策略。

关键词：连铸切割；在线决策；字典序优化；动态规划；最小费用流

## 一、问题重述

### 1.1 背景与已知条件

连铸钢坯以 1.0 米/分钟匀速通过切割机。每次切割从固定工作起点开始，切割机随钢坯移动 3 分钟，完成切断后用 1 分钟返回。结晶器中心到工作起点的钢坯长度为 60.0 米；异常发生时，切割工序立即获知，结晶器内长 0.8 米的报废段不能进入后续产品 [1]。

一次切下的钢坯必须长 4.8—12.6 米，才能被运走；下道工序接受 8.0—11.6 米的钢坯，还需满足用户给出的更窄目标范围。偏长钢坯可离线修剪，修剪部分报废；含缺陷钢坯也先整体切下，再离线回收其中符合要求的连续健康部分。题目要求优先减小报废长度，再尽量接近用户目标长度。

### 1.2 逐问任务

**问题一：**用户目标 9.5 米、范围 9.0—10.0 米，针对题目列出的 12 种尾坯长度，给出最优切割方案与损失。

**问题二：**异常时刻为 0.0、45.6、98.6、131.5、190.8、233.3、266.0、270.7 及 327.9 分钟，目标及范围同问题一。针对每次通知给出新段方案、当前段调整或不调整，以及相应损失。

**问题三：**沿用问题二的通知时刻与实时要求，分别将目标改为 8.5 米（范围 8.0—9.0 米）和 11.1 米（范围 10.6—11.6 米），给出两组完整方案。

## 二、问题分析

### 2.1 问题一的分析

尾坯总长已知，没有未来信息问题。关键是同时处理合格品与废料的运输：末端不足 4.8 米不能直接当成允许丢弃的独立一次切割件。先确定合格品件数和整块报废件数，可将连续长度优化化为有限计数枚举，再利用凸性分配合格长度。

### 2.2 问题二的分析

异常通知比缺陷到达切割点早约 60 分钟，提供了调整尚未执行切口的时间。已启动的切割不能撤回；把所有 9 个异常提前输入一次离线优化，会改变题目。每次通知时应保存真实执行状态，仅对已知缺陷及其后健康延续规划，待新通知到达后再重算。

一次切割最长 12.6 米，小于两个最短 8.0 米合格品的总长，因此每块一次切割件至多回收一个合格品。这使每条切割边的最大回收量可以直接计算，从而用无环图上的动态规划处理两级目标，无需为每块材料建立复杂的多件装箱模型。

## 2.3 问题三的分析

更换目标区间会改变健康间隔能容纳的产品数，影响并非简单按目标长度比例缩放。例如两次相邻异常之间若只有 3.9 米健康材料，三种目标均无法利用；而 31.9 米健康材料可容纳三块 10.6 米以上产品，却放不下四块 8.0 米产品。应复用同一算法重新计算，而不是直接改写问题二的结果。

## 三、模型假设

1. 切缝宽度取零，离线加工不设额外时间及能力约束；题面未提供这些参数。若实际切缝或离线资源不可忽略，应将其长度损失和能力限制加入模型。

2. 0.8 米报废段以结晶器中心为中心。以材料到达切割起点的时刻为坐标，则时刻  $t$  通知的报废区间为  $[t+59.6, t+60.4]$  米。图示未规定其他偏移，中心对齐属于明确采用的解释。

3. 初始状态取  $t=0$  时坐标 0 处的切割已经启动，该边界不能更改。同一时刻的新通知先于新的切割启动处理。初始相位原题未给定，下文报告半个目标长度的相位变更结果。

4. 严格将 4.8—12.6 米限制用于每块一次切割件，包括尾坯余段；离线修剪废料不受此运输下限约束。产品必须同时满足基本要求和正常要求。

5. 第二目标取合格产品长度偏差平方和；报废件不计入合格产品偏差。题面没有唯一指定偏差范数，因此这是一项可替换的偏好定义。在线规划采用 0.1 米网格，另用 0.05 米网格检验离散影响。

#### 四、符号说明

| 符号         | 含义                 | 单位               |
|------------|--------------------|------------------|
| $L$        | 尾坯总长度              | m                |
| $a, b$     | 一次切割长度上下限 4.8、12.6 | m                |
| $l, u, d$  | 产品区间下限、上限与目标       | m                |
| $p_i, r_i$ | 第 i 块一次切割长度与回收长度   | m                |
| $W, V$     | 总报废长度与合格长度偏差平方和    | m、m <sup>2</sup> |
| $s, t$     | 最近锁定切口与当前通知时刻      | m、min            |
| $D_j$      | 第 j 段已知报废区间        | m                |
| $h$        | 在线切口网格间距           | m                |

#### 五、模型建立与求解

##### 5.1 问题一：尾坯的连续长度优化

设产生  $n$  个合格品、 $m$  个整块报废件。合格品长度在  $[l, u]$  中；相应一次切割件可比合格品更长，但不超过  $b$ 。可行计数满足

$$nl + ma \leq L \leq (n + m)b, \quad n, m \in \mathbb{Z}_{\geq 0}, \quad n + m \geq 1. \quad (1)$$

对固定件数，产品总长度不能超过  $n$  个上限，也不能占用整块报废件所需的最短运输长度。因此最大可回收总量及损失为

$$R_{n,m} = \min\{nu, L - ma\}, \quad W_{n,m} = L - R_{n,m}, \quad R_{n,m} \geq nl. \quad (2)$$

在固定  $R_{n,m}$  下，平方偏差具有凸性。把每件合格品分配为  $R_{n,m}/n$  可达到最小二级目标； $n = 0$  时约定该目标为 0。

$$V_{n,m} = n \left( \frac{R_{n,m}}{n} - d \right)^2 \quad (n > 0). \quad (3)$$

枚举  $n + m \leq \lfloor L/a \rfloor$ ，按  $(W, V)$  字典序比较：只有损失相同时才比较长度偏差。取最少一次切割件数作为完全并列时的显示选择。给定回收长度后，将剩余材料分配给各一次切割件，容量均不超过  $b$ ；上述总容量条件保证这种分配存在。本题用有理数计算，避免小数舍入改变质量守恒。

表 1 列出全部尾坯。乘号表示连续相同长度的件数；分数保留精确长度，合格品列不包含整块报废件。一次切割件与合格品的长度差即离线废料。

| 尾坯<br>(m) | 一次切割方案<br>(m)                                   | 合格品<br>(m)                   | 损失<br>(m) |
|-----------|-------------------------------------------------|------------------------------|-----------|
| 109.0     | $11 \times (\frac{109}{11})$                    | $11 \times (\frac{109}{11})$ | 0.0       |
| 93.4      | $10 \times (\frac{467}{50})$                    | $10 \times (\frac{467}{50})$ | 0.0       |
| 80.9      | $\frac{109}{10} + 7 \times (10)$                | $8 \times (10)$              | 0.9       |
| 72.0      | $8 \times (9)$                                  | $8 \times (9)$               | 0.0       |
| 62.7      | $\frac{63}{5} + \frac{101}{10} + 4 \times (10)$ | $6 \times (10)$              | 2.7       |
| 52.5      | $\frac{25}{2} + 4 \times (10)$                  | $5 \times (10)$              | 2.5       |
| 44.9      | $\frac{63}{5} + \frac{123}{10} + 2 \times (10)$ | $4 \times (10)$              | 4.9       |
| 42.7      | $\frac{63}{5} + \frac{101}{10} + 2 \times (10)$ | $4 \times (10)$              | 2.7       |
| 31.6      | $\frac{58}{5} + 2 \times (10)$                  | $3 \times (10)$              | 1.6       |
| 22.7      | $\frac{63}{5} + \frac{101}{10}$                 | $2 \times (10)$              | 2.7       |
| 14.5      | $\frac{97}{10} + \frac{24}{5}$                  | $\frac{97}{10}$              | 4.8       |
| 13.7      | $\frac{89}{10} + \frac{24}{5}$                  | 无                            | 13.7      |

尾坯损失依次为 0.0、0.0、0.9、0.0、2.7、2.5、4.9、2.7、1.6、2.7、4.8、13.7 米。

13.7 米这一边界例尤其重要：若要得到一件至少 9.0 米产品，剩余最多 4.7 米，不能独立运走；又不能把 13.7 米整块切下，因为超过 12.6 米上限。两块均不足 9.0 米时，尽管其中一块可能达到基本工序的 8.0 米下限，仍不能满足本问正常要求。14.5 米则可分为 9.7 与 4.8 米，回收前者。

## 5.2 问题二：异常通知下的在线规划

令每个网格切口为图节点  $x$ ，满足  $a \leq y - x \leq b$  时可从  $x$  连边至  $y$ 。该边表示一次切下区间  $[x, y]$ 。从此区间扣除当前已知报废段后，取最长连续健康部分长度  $c(x, y)$ ，其最大回收量为

$$r(x, y) = \begin{cases} \min\{u, c(x, y)\}, & c(x, y) \geq l, \\ 0, & c(x, y) < l. \end{cases} \quad (4)$$

不能把报废段两侧不相连的健康长度相加；两侧各 5 米，即便总计 10 米，也不能得到一块 9 米以上的连续产品。边的两级代价分别为一次切割长度减去回收量，以及回收长度的平方偏差：

$$w(x, y) = y - x - r(x, y), \quad v(x, y) = \begin{cases} (r(x, y) - d)^2, & r(x, y) > 0, \\ 0, & r(x, y) = 0. \end{cases} \quad (5)$$

通知到达时，以最近已经启动切割的坐标  $s$  为起点。首个新切口必须不早于当前通知时刻；其他边严格向右，不会回到已经经过的材料位置。题设速度为 1 米/分钟，4.8 米最短运输长度对应至少 4.8 分钟切割间隔，大于 3 分钟切割加 1 分钟返回，因此运输下限已蕴含本题的机器周期限制。验证程序仍单独核对两者。

$$F(y) = \min_{x \rightarrow y}^{\text{lex}} \{F(x) + (w(x, y), v(x, y))\}, \quad F(s) = (0, 0). \quad (6)$$

令  $z$  为最后一个已知报废段的结束位置，规划在第一个不小于  $z$  的切口终止。因为到该切口的一次切割最长  $b$ ，候选终点只需考虑  $[z, z+b]$ ；之后可恢复目标定长切割，直到新的异常通知。所有候选终点也按两级目标比较，完全并列时再依次选更少切口和更早终点。有限无环图的递推给出当前已知信息与网格约束下的精确最优方案。

每次通知先执行上一方案中启动时间严格早于本次通知的切口，将这些切口永久保留；再加入新报废区间，只重算剩余部分。同一异常序列的任意前缀，必须与在更长序列中运行到该时刻的决策一致。全部异常仅用于最后的事后下界计算，不传入尚未到达的在线决策。

问题二累计实际报废为 23.8 米。以下表中的方案均从“已锁定”切口向后依次列一次切割长度，顿号分隔、乘号合并连续等长件；“余下损失”是该时刻已知缺陷下的新待执行方案损失，不是当次新增损失，不能按行相加。下一刀“不变”表示最近的尚未启动切口不调整，其后的切口仍可重排。第一行给出首次方案，后续行保留修改前后的待切方案。

### 5.2.1 问题二事件方案：第 1—3 次通知

| 通知<br>(min; 下一刀) | 已锁定<br>(m) | 原待切长度<br>(m) | 新待切长度<br>(m)               | 余下损失<br>(m) |
|------------------|------------|--------------|----------------------------|-------------|
| 0.0; 首次          | 0.0        | 首次通知, 无前案    | 4×9.9、10.0、10.8            | 0.8         |
| 45.6; 不变         | 39.6       | 10.0、10.8    | 2×10.0、10.8、10.4、2×12.6    | 6.4         |
| 98.6; 调整         | 93.4       | 12.6         | 10.0、12.6、2×10.0、10.4、12.6 | 5.6         |

### 5.2.2 问题二事件方案：第 4—6 次通知

| 通知<br>(min; 下一刀) | 已锁定<br>(m) | 原待切长度<br>(m)   | 新待切长度<br>(m)               | 余下损失<br>(m) |
|------------------|------------|----------------|----------------------------|-------------|
| 131.5; 不变        | 126.0      | 10.0、10.4、12.6 | 2×10.0、10.4、12.6、10.3、12.6 | 5.9         |
| 190.8; 调整        | 179.3      | 12.6           | 11.5、10.8、2×9.7、2×9.8、10.6 | 3.4         |
| 233.3; 不变        | 230.8      | 9.8、10.6       | 2×9.8、10.8、2×10.0、12.5     | 3.3         |

5.2.3 问题二事件方案：第 7—9 次通知

| 通知<br>(min; 下一刀) | 已锁定<br>(m) | 原待切长度<br>(m)              | 新待切长度<br>(m)               | 余下损失<br>(m) |
|------------------|------------|---------------------------|----------------------------|-------------|
| 266.0; 不变        | 261.2      | 2×10.0、12.5               | 3×10.0、12.5、<br>10.1、12.6  | 5.2         |
| 270.7; 不变        | 261.2      | 3×10.0、12.5、<br>10.1、12.6 | 3×10.0、12.5、<br>2×10.0、7.4 | 9.9         |
| 327.9; 调整        | 323.7      | 7.4                       | 4.8、12.0、<br>4×9.4、10.2    | 8.2         |

5.3 问题三：替换产品区间后的决策

问题三两组累计实际报废分别为 16.0 米和 25.9 米。沿用同一状态、回收定义和信息边界，只替换 l、u、d，分别得到下列方案。每块一次切割件的绝对起止位置及离线回收区间另由 event\_plans.csv 完整列出，可逐行核对正文长度表。

在 266.0 与 270.7 分钟的两处异常之间，健康材料仅长  $4.7-0.8=3.9$  米，三种目标都不能回收。第二次通知时，这两段缺陷还在上游，相关切口尚可调整。图 1 以 11.1 米目标为例展示局部修改；橙色的新报废段在旧方案形成时尚未知，旧方案不是已经执行的错误切割。

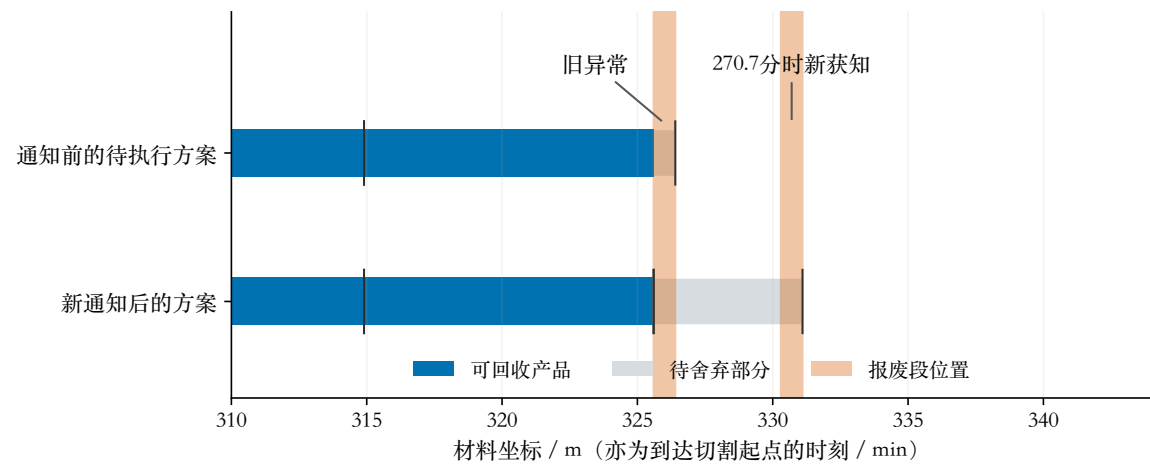


图 1 相邻异常附近的待切方案调整。黑色短线为一次切口，蓝色为可回收区间，灰色为舍弃部分；两条均为局部截取。



### 5.3.1 目标 8.5 米：第 1—3 次通知

| 通知<br>(min; 下一刀) | 已锁定<br>(m) | 原待切长度<br>(m) | 新待切长度<br>(m)                  | 余下损失<br>(m) |
|------------------|------------|--------------|-------------------------------|-------------|
| 0.0; 首次          | 0.0        | 首次通知, 无前案    | 6×8.5、9.4                     | 0.8         |
| 45.6; 不变         | 42.5       | 8.5、9.4      | 8.5、8.6、9.7、<br>8.9、2×9.0、9.8 | 1.6         |
| 98.6; 调整         | 96.2       | 9.8          | 9.0、9.5、<br>4×8.7、9.5         | 1.6         |

### 5.3.2 目标 8.5 米：第 4—6 次通知

| 通知<br>(min; 下一刀) | 已锁定<br>(m) | 原待切长度<br>(m) | 新待切长度<br>(m)                | 余下损失<br>(m) |
|------------------|------------|--------------|-----------------------------|-------------|
| 131.5; 不变        | 123.4      | 3×8.7、9.5    | 4×8.7、8.8、<br>2×8.0、8.9     | 1.6         |
| 190.8; 调整        | 183.0      | 8.9          | 8.1、9.1、2×8.3、<br>3×8.4、9.2 | 1.6         |
| 233.3; 不变        | 225.2      | 2×8.4、9.2    | 3×8.4、9.1、<br>2×8.3、8.4、9.2 | 1.6         |

### 5.3.3 目标 8.5 米：第 7—9 次通知

| 通知<br>(min; 下一刀) | 已锁定<br>(m) | 原待切长度<br>(m)                | 新待切长度<br>(m)                     | 余下损失<br>(m) |
|------------------|------------|-----------------------------|----------------------------------|-------------|
| 266.0; 不变        | 259.5      | 2×8.3、8.4、9.2               | 2×8.3、2×8.4、<br>9.8、11.1、12.6    | 6.5         |
| 270.7; 不变        | 267.8      | 8.3、2×8.4、<br>9.8、11.1、12.6 | 8.3、2×8.4、9.8、<br>2×9.0、10.4     | 11.2        |
| 327.9; 调整        | 320.7      | 10.4                        | 7.2、11.2、<br>2×8.0、3×8.1、<br>8.9 | 11.2        |

#### 5.3.4 目标 11.1 米：第 1—3 次通知

| 通知<br>(min; 下一刀) | 已锁定<br>(m) | 原待切长度<br>(m) | 新待切长度<br>(m)                       | 余下损失<br>(m) |
|------------------|------------|--------------|------------------------------------|-------------|
| 0.0; 首次          | 0.0        | 首次通知, 无前案    | 2×11.6、12.0、<br>2×12.6             | 2.4         |
| 45.6; 调整         | 35.2       | 2×12.6       | 11.6、12.2、<br>12.6、2×11.2、<br>12.0 | 2.8         |
| 98.6; 调整         | 94.0       | 12.0         | 11.2、4.8、11.6、<br>12.2、2×12.6      | 7.4         |

#### 5.3.5 目标 11.1 米：第 4—6 次通知

| 通知<br>(min; 下一刀) | 已锁定<br>(m) | 原待切长度<br>(m) | 新待切长度<br>(m)                       | 余下损失<br>(m) |
|------------------|------------|--------------|------------------------------------|-------------|
| 131.5; 调整        | 121.6      | 12.2、2×12.6  | 2×11.6、12.3、<br>12.6、10.7、11.5     | 3.4         |
| 190.8; 调整        | 180.4      | 11.5         | 10.7、12.4、<br>2×11.6、11.9、<br>12.6 | 2.1         |
| 233.3; 调整        | 226.7      | 11.9、12.6    | 2×11.6、6.0、<br>3×12.6              | 9.0         |

### 5.3.6 目标 11.1 米：第 7—9 次通知

| 通知<br>(min; 下一刀) | 已锁定<br>(m) | 原待切长度<br>(m)              | 新待切长度<br>(m)                  | 余下损失<br>(m) |
|------------------|------------|---------------------------|-------------------------------|-------------|
| 266.0; 调整        | 255.9      | 3×12.6                    | 2×11.6、2×12.6、<br>10.6、11.5   | 3.8         |
| 270.7; 不变        | 267.5      | 11.6、2×12.6、<br>10.6、11.5 | 11.6、2×12.6、<br>10.6、10.7、5.5 | 8.5         |
| 327.9; 调整        | 325.6      | 5.5                       | 4.8、11.9、<br>3×11.3、12.1      | 6.3         |

## 六、验证与灵敏度分析

### 6.1 独立网络流复算与连续下界

独立程序先把材料离散为健康/报废单元，逐单元扫描每条边中的最长健康连段，未调用主程序的区间回收函数。以单位流守恒构造最小费用流线性规划，先最小化损失，再固定最小损失求最小偏差，使用 HiGHS 求解。无环图的网络流可行域具有整数顶点；程序同时核对流量整数性与守恒残差。12 种尾坯损失，以及 27 次在线决策的损失与偏差，均与主算法一致。尾坯连续偏差另按有理数恒等式核对，未将网格偏差冒充连续偏差。

为判断总损失是否仍受网格限制，事后使用完整已实现异常序列构造连续下界。对长度  $\ell_j$  的有限健康间隔，最多形成  $\lfloor \ell_j/l \rfloor$  个合格品，总回收量不超过  $\min\{\ell_j, u\lfloor \ell_j/l \rfloor\}$ 。再加上报废段本身，得

$$W_{\text{LB}} = \sum_j |D_j| + \sum_j \left[ \ell_j - \min \left\{ \ell_j, u \left\lfloor \frac{\ell_j}{l} \right\rfloor \right\} \right]. \quad (7)$$

该下界放宽了运输、机器周期和在线限制，因此即使允许连续切口并提前知道全部异常，也不可能低于它。初始健康区间从声明的初始切口开始计算；最后报废段后的无限健康延续无需额外损失。

| 目标<br>(m) | 在线实际损失<br>(m) | 连续下界<br>(m) | 是否达到 |
|-----------|---------------|-------------|------|
| 9.5       | 23.8          | 23.8        | 是    |
| 8.5       | 16.0          | 16.0        | 是    |
| 11.1      | 25.9          | 25.9        | 是    |

三组损失均达到下界，故第一目标在本初始状态和给定序列下已经最优。这不能推出该策略对任何未来序列均具有离线最优表现，也不能推出整个已实现序列的二级偏差全局最优；后者只在每次已知信息与当前网格的规划问题中得到最优性验证。

## 6.2 基线、网格与初始相位

固定目标长度切割作为基线，其切口完全不依据异常调整，仅在切下后尽量离线回收。它与在线策略均处理到最后报废段后的一个完整切口；多出的健康目标定长件不增加损失。基线与本文方案的差别反映调整切口的价值，不是换一个算法名称。

| 目标<br>(m) | 定长基线损失<br>(m) | 在线损失<br>(m) | 0.1 米偏差<br>(m <sup>2</sup> ) | 0.05 米偏差<br>(m <sup>2</sup> ) |
|-----------|---------------|-------------|------------------------------|-------------------------------|
| 9.5       | 76.0          | 23.8        | 6.340                        | 6.315                         |
| 8.5       | 68.0          | 16.0        | 4.670                        | 4.610                         |
| 11.1      | 88.8          | 25.9        | 5.600                        | 5.590                         |

网格加密至 0.05 米后，三种累计损失均不变，但长度偏差有所下降。这与损失已达到连续下界一致，也说明二级偏差仍有离散影响。将初始已启动切口移至负半个目标长度，所得损失如下：

| 目标<br>(m) | 初始切口<br>(m) | 变更后损失<br>(m) | 该状态下的下界<br>(m) |
|-----------|-------------|--------------|----------------|
| 9.5       | -4.75       | 23.80        | 23.80          |
| 8.5       | -4.25       | 16.85        | 16.85          |
| 11.1      | -5.55       | 24.30        | 24.30          |

初始相位改变了第一段可用健康材料的长度，不能把不同状态下的损失当成同一问题的相互矛盾答案。原题未给该状态，因此实际部署应读取最近已启动切口，而不是固定套用本文数值。0.1 米网格下 27 次规划的本机最长耗时为 0.042 秒；计时不含事后独立 LP 验证，不构成设备实时性的硬保证。

## 七、模型评价与改进

模型利用一次运输件最多容纳一个合格产品的结构，避免多件装箱的额外组合变量；用字典序比较直接表达题目优先级，避免任意加权把少报废与接近目标混为一谈。在线执行记录保存已启动切口与每次通知前后的方案，能够检验是否偷用了未来信息。独立网络流与连续下界分别核对网格两级目标和连续总损失。

主要边界是初始切口、缺陷中心对齐、零切缝和离线能力无限。实际切缝会改变每刀的质量守恒，离线容量可能使即时损失最小方案难以执行；需接入实际设备状态再规划。当前次优先级取平方偏差，其他用户偏好需重新求解。本文未给出所有可能异常序列下的竞争比，也未证明二级目标的连续最优性。

## 八、结论

在声明的初始状态及加工假设下，本文完成 12 种尾坯和三种目标下各 9 次异常决策。9.5、8.5、11.1 米目标的在线累计损失分别为 23.8、16.0、25.9 米，均达到相应连续材料下界。方程、逐次方案与独立计算共同支撑这些条件结论；对初始相位与次优先级的边界已经明确。

## AI 工具使用声明

本训练稿使用 AI 工具参与题意分析、模型推导、代码编写、验证和文字排版。尚未完成参赛团队的人工审核、正式 AI 使用详情与比赛提交材料审查，不能作为已经获准提交的论文。

## 参考文献

[1] 全国大学生数学建模竞赛组委会. 2021 年高教社杯全国大学生数学建模竞赛 D 题：连铸切割的在线优化 [Z]. 2021. 原题两页及图 1、图 2；使用本项目冻结的原始 PDF。

## 附录：支撑文件与完整源程序

下列文件与支撑包使用同一份源码，逐文件列出完整程序。独立复算命令为 `python3 -I reproduce.py -output reproduced`；可选 `-paper` 重新生成图表和论文。复算从已核对的 `facts.json` 参数开始，不自动重新解释原题。

| 文件                    | 类别       | 字节数    |
|-----------------------|----------|--------|
| LICENSE               | document | 1066   |
| README.md             | document | 2159   |
| artifact_io.py        | code     | 877    |
| environment.json      | data     | 149    |
| expected-results.json | data     | 148152 |
| facts.json            | data     | 2000   |
| paper.py              | code     | 23244  |
| plots.py              | code     | 3375   |
| render_latex.py       | code     | 11811  |
| reproduce.py          | code     | 5684   |
| requirements.txt      | document | 74     |
| sensitivity.py        | code     | 2319   |
| solve.py              | code     | 9022   |
| verify.py             | code     | 10323  |
| manifest.json         | 文件指纹清单   | 随包生成   |

artifact\_io.py

```
"""Small portable artifact I/O; no dependency on the repository workflow
engine."""
import hashlib
import json
from pathlib import Path
```

```

def read_json(path):
    path = Path(path)
    data = json.loads(path.read_text(encoding="utf-8"))
    if not isinstance(data, dict):
        raise ValueError(f"Expected a JSON object in {path}")
    return data

def write_json(path, data):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    temporary.write_text(json.dumps(data, ensure_ascii=False, indent=2,
allow_nan=False) + "\n", encoding="utf-8")
    temporary.replace(path)

def sha256_file(path):
    digest = hashlib.sha256()
    with Path(path).open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b''):
            digest.update(chunk)
    return digest.hexdigest()

```

paper.py

```

"""Write the three-question manuscript from computed, independently
checked artifacts."""

```

```

import argparse
import hashlib
import json
import zipfile
from fractions import Fraction
from itertools import groupby
from pathlib import Path
from solve import read_json, write_json, sha256_file

```

```

def expression(values):
    parts=[]
    for value,group in groupby(map(Fraction,values)):
        count=len(list(group))

```

```

        base=str(value.numerator) if value.denominator==1 else
rf"\frac{{{value.numerator}}}{{{{value.denominator}}}}"
        parts.append((str(count)+r"\times("+base+")") if count>1 else
base)
    return "$"+"+".join(parts)+"$" if parts else " 无"

```

```

def plan_text(pieces):
    if not pieces: return " 首次通知, 无前案"
    parts=[]
    for length,group in groupby(edge["end"]-edge["start"] for edge in
pieces):
        count=len(list(group)); value=f"{length/10:.1f}"
        parts.append((str(count)+"x" if count>1 else "")+value)
    return "\ ".join(parts)

```

```

def support_appendix(run, support_root=None):
    archive = None
    if support_root is None:
        folder = run / "artifacts/submission-package"
        receipt = read_json(folder / "package_receipt.json")
        if receipt["sha256"] != sha256_file(folder / "support.zip"):
            raise ValueError("support ZIP changed after packaging")
        archive = zipfile.ZipFile(folder / "support.zip")
        read = archive.read
    else:
        root = Path(support_root).resolve()
        from reproduce import check_manifest
        check_manifest(root)
        read = lambda name: (root / name).read_bytes()
    try:
        manifest = json.loads(read("manifest.json"))
        blocks = [{"text": "下列文件与支撑包使用同一份源码, 逐文件列出完整程序。"}]

```

独立复算命令为python3 -I reproduce.py --output reproduced;

可选--paper重新生成图表和论文。复算从已核对的facts.json参数开始, 不自动重新解释原题。

"},

```

        {"table": [{"文件", "类别", "字节数"}] + [[item["path"],
item["role"], item["bytes"]] for item in manifest["files"]] +
[["manifest.json", "文件指纹清单", "随包生成"]]}]
        for item in manifest["files"]:
            content = read(item["path"])

```



```

        if hashlib.sha256(content).hexdigest() != item["sha256"]:
            raise ValueError("support source content differs: " +
item["path"])
        if item["role"] == "code":
            blocks.append({"code": content.decode("utf-8-sig"),
"filename": item["path"], "source_sha256": item["sha256"]})
        return {"id": "support-appendix", "title": "附录：
支撑文件与完整源程序", "page_break_before": True, "blocks": blocks}
    finally:
        if archive is not None:
            archive.close()

def compose(run, support_root=None):
    data=read_json(run/"artifacts/solution.json")
    verified=read_json(run/"artifacts/independent.json")
    sensitivity=read_json(run/"artifacts/sensitivity.json")
    digest=sha256_file(run/"artifacts/solution.json")
    if verified["solution_sha256"]!=digest or
sensitivity["solution_sha256"]!=digest or not verified["passed"]:
        raise ValueError("stale or failed numerical evidence")
    sections=[]
    text=lambda s:{"text":s}
    eq=lambda s:{"equation":s}
    table=lambda rows:{"table":rows}
    def section(title,blocks,level=1,page=False,key=None):
        sections.append({"title":title,"blocks":blocks,"level":level,"pa
ge_break_before":page,"id":key})
        losses={k:v["total_waste_m"] for k,v in data["online"].items()}
        max_seconds=max(r["planning_seconds"] for v in
data["online"].values() for r in v["events"])
        section(" 摘要",[
            text("针对连铸尾坯处理与异常通知下的在线切割问题，
本文将一次切割的运输长度限制与离线回收的产品长度要求分开建模，以报废总长度为第一目标、
合格产品偏离目标长度的平方和为第二目标，
形成尾坯解析分配与事件驱动动态规划相结合的求解方法。"),
            {"lead":"针对问题一：","text":"枚举合格产品数及整块报废件数，
推导每种计数下的最大回收量，再由凸性确定等长分配。给出全部12种尾坯方案；
其中109.0米和93.4米均可零损失回收，14.5米可回收9.7米。
严格执行尾坯也需满足4.8米运输下限时，13.7米尾坯没有合格产品可回收。"},

```

```

{"lead": "针对问题二: ", "text": f"按异常通知逐次更新已知报废区间,
锁定已启动的切口, 在0.1米网格上用字典序最短路重新规划。9.5米目标下,
给定9次异常序列的最终损失为{losses['Q2']:.1f}米, 达到连续材料分割理论下界。"},
{"lead": "针对问题三: ", "text": f"保持信息边界和求解方法, 仅替换产品区间。
8.5米与11.1米目标下,
最终损失分别为{losses['Q3a']:.1f}米和{losses['Q3b']:.1f}米, 也达到相应下界。
三组结果均给出原待执行方案、新方案和剩余计划损失。"},
text(f"独立以单元占用图构造两阶段最小费用流线性规划,
12种尾坯损失及27次事件的两级目标均与主算法一致。实测单次规划最多{max_seconds:.3f}秒;
网格加密不改变三组损失, 但会改善长度偏差。改变初始切口相位可改变最小损失,
因此最优性结论明确限定初始状态、零切缝和所声明的二级偏好,
不外推为未知未来异常下的普遍最优策略。"),
text("关键词: 连铸切割; 在线决策; 字典序优化; 动态规划;
最小费用流"), key="abstract")
section(" 一、问题重述", [], page=True)
section("1.1 背景与已知条件", [
text("连铸钢坯以1.0米/分钟匀速通过切割机。每次切割从固定工作起点开始,
切割机随钢坯移动3分钟, 完成切断后用1分钟返回。结晶器中心到工作起点的钢坯长度为60.0米;
异常发生时, 切割工序立即获知, 结晶器内长0.8米的报废段不能进入后续产品[1]。"),
text("一次切下的钢坯必须长4.8—12.6米, 才能被运走; 下道工序接受8.0—
11.6米的钢坯, 还需满足用户给出的更窄目标范围。偏长钢坯可离线修剪, 修剪部分报废;
含缺陷钢坯也先整体切下, 再离线回收其中符合要求的连续健康部分。题目要求优先减小报废长度,
再尽量接近用户目标长度。")], 2)
section("1.2 逐问任务", [
{"lead": "问题一: ", "text": "用户目标9.5米、范围9.0—10.0米,
针对题目列出的12种尾坯长度, 给出最优切割方案与损失。"},
{"lead": "问题二: ", "text": "异常时刻为0.0、45.6、98.6、131.5、190.8、
233.3、266.0、270.7及327.9分钟, 目标及范围同问题一。针对每次通知给出新段方案、
当前段调整或不调整, 以及相应损失。"},
{"lead": "问题三: ", "text": "沿用问题二的通知时刻与实时要求,
分别将目标改为8.5米(范围8.0—9.0米)和11.1米(范围10.6—11.6米), 给出两组完整方案。
"}], 2)
section(" 二、问题分析", [])
section("2.1 问题一的分析", [
text("尾坯总长已知, 没有未来信息问题。关键是同时处理合格品与废料的运输:
末端不足4.8米不能直接当成允许丢弃的独立一次切割件。先确定合格品件数和整块报废件数,
可将连续长度优化化为有限计数枚举, 再利用凸性分配合格长度。")], 2)
section("2.2 问题二的分析", [
text("异常通知比缺陷到达切割点早约60分钟, 提供了调整尚未执行切口的时间。
已启动的切割不能撤回; 把所有9个异常提前输入一次离线优化, 会改变题目。
每次通知时应保存真实执行状态, 仅对已知缺陷及其后健康延续规划, 待新通知到达后再重算。"),

```

text("一次切割最长12.6米，小于两个最短8.0米合格品的总长，因此每块一次切割件至多回收一个合格品。这使每条切割边的最大回收量可以直接计算，从而用无环图上的动态规划处理两级目标，无需为每块材料建立复杂的多件装箱模型。"))],2)

section("2.3 问题三的分析",[

text("更换目标区间会改变健康间隔能容纳的产品数，影响并非简单按目标长度比例缩放。例如两次相邻异常之间若只有3.9米健康材料，三种目标均无法利用；而31.9米健康材料可容纳三块10.6米以上产品，却放不下四块8.0米产品。应复用同一算法重新计算，而不是直接改写问题二的结果。"))],2)

section(" 三、模型假设",[

text("1. 切缝宽度取零，离线加工不设额外时间及能力约束；题面未提供这些参数。若实际切缝或离线资源不可忽略，应将其长度损失和能力限制加入模型。"),

text("2. 0.8米报废段以结晶器中心为中心。以材料到达切割起点的时刻为坐标，则时刻t通知的报废区间为[t+59.6,t+60.4]米。图示未规定其他偏移，中心对齐属于明确采用的解释。"),

text("3. 初始状态取t=0时坐标0处的切割已经启动，该边界不能更改。同一时刻的新通知先于新的切割启动处理。初始相位原题未给定，下文报告半个目标长度的相位变更结果。"),

text("4. 严格将4.8—12.6米限制用于每块一次切割件，包括尾坯余段；离线修剪废料不受此运输下限约束。产品必须同时满足基本要求和正常要求。"),

text("5. 第二目标取合格产品长度偏差平方和；报废件不计入合格产品偏差。题面没有唯一指定偏差范数，因此这是一项可替换的偏好定义。在线规划采用0.1米网格，另用0.05米网格检验离散影响。"))]

section("四、符号说明",[table([["符号","含义","单位"],["\$L\$","尾坯总长度","m"],["\$a,b\$","一次切割长度上下限4.8、12.6","m"],["\$l,u,d\$","产品区间下限、上限与目标","m"],["\$p\_{i,r\_i}\$","第i块一次切割长度与回收长度","m"],["\$W,V\$","总报废长度与合格长度偏差平方和","m、m<sup>2</sup>"],["\$s,t\$","最近锁定切口与当前通知时刻","m、min"],["\$D\_j\$","第j段已知报废区间","m"],["\$h\$","在线切口网格间距","m"]]))]

section(" 五、模型建立与求解",[

section("5.1 问题一：尾坯的连续长度优化",[

text("设产生n个合格品、m个整块报废件。合格品长度在[l,u]中；相应一次切割件可比合格品更长，但不超过b。可行计数满足"),

$$eq(r"n\leq \frac{L-mb}{l}, \quad n,m\in\mathbb{Z}_{\geq 0}, \quad n+m\geq 1.")$$

text("对固定件数，产品总长度不能超过n个上限，也不能占用整块报废件所需的最短运输长度。因此最大可回收总量及损失为"),

$$eq(r"R_{n,m}=\min\{nu, L-ma\}, \quad W_{n,m}=L-R_{n,m}, \quad R_{n,m}\geq nl.")$$

text("在固定R<sub>n,m</sub>下，平方偏差具有凸性。把每件合格品分配为R<sub>n,m</sub>/n可达到最小二级目标；n=0时约定该目标为0。"),

$$eq(r"V_{n,m}=n\left(\frac{R_{n,m}}{n}-d\right)^2 \quad (n>0).")$$

text(r"枚举 $n+m \leq \lfloor L/a \rfloor$ ，按 $(W,V)$ 字典序比较：  
只有损失相同时才比较长度偏差。取最少一次切割件数作为完全并列时的显示选择。  
给定回收长度后，将剩余材料分配给各一次切割件，容量均不超过 $b$ ；  
上述总容量条件保证这种分配存在。本题用有理数计算，避免小数舍入改变质量守恒。"),

text("表1列出全部尾坯。乘号表示连续相同长度的件数；分数保留精确长度，  
合格品列不包含整块报废件。一次切割件与合格品的长度差即离线废料。"),

table([["尾坯/m", "一次切割方案/m", "合格品/m", "损失/m"]]+[[f"{r['tail\_']:.1f}", expression(r["exact"] ["primary"]), expression([v for v in r["exact"] ["recovered"] if Fraction(v))], f"{r['waste\_m']:.1f}"] for r in data["tails"]]),

text("尾坯损失依次为"+"、".join(f"{r['waste\_m']:.1f}" for r in data["tails"])+ "米。"),

text("13.7米这一边界例尤其重要：若要得到一件至少9.0米产品，剩余最多4.7米，  
不能独立运走；又不能把13.7米整块切下，因为超过12.6米上限。两块均不足9.0米时，  
尽管其中一块可能达到基本工序的8.0米下限，仍不能满足本问正常要求。  
14.5米则可分为9.7与4.8米，回收前者。")], 2, key="q1")

section("5.2 问题二：异常通知下的在线规划", [

text(r"令每个网格切口为图节点 $x$ ，满足 $a \leq y-x \leq b$ 时可从 $x$ 连边至 $y$ 。  
该边表示一次切下区间 $[x,y]$ 。从此区间扣除当前已知报废段后，  
取最长连续健康部分长度 $c(x,y)$ ，其最大回收量为"),

eq(r"r(x,y)=\begin{cases} \min\{u, c(x,y)\}, & c(x,y) \geq l, \\ 0, & c(x,y) < l. \end{cases}"),

text("不能把报废段两侧不相连的健康长度相加；两侧各5米，即便总计10米，  
也不能得到一块9米以上的连续产品。边的两级代价分别为一次切割长度减去回收量，  
以及回收长度的平方偏差："),

eq(r"w(x,y)=y-x-r(x,y), \quad v(x,y)=\begin{cases} (r(x,y)-d)^2, & r(x,y) > 0, \\ 0, & r(x,y) = 0. \end{cases}"),

text("通知到达时，以最近已经启动切割的坐标 $s$ 为起点。  
首个新切口必须不早于当前通知时刻；其他边严格向右，不会回到已经经过的材料位置。  
题设速度为1米/分钟，4.8米最短运输长度对应至少4.8分钟切割间隔，  
大于3分钟切割加1分钟返回，因此运输下限已蕴含本题的机器周期限制。验证程序仍单独核对两者。"),

eq(r"F(y)=\min\_{x \text{ to } y} \{F(x)+(w(x,y), v(x,y))\}, \quad F(s)=(0,0)."),

text("令 $z$ 为最后一个已知报废段的结束位置，规划在第一个不小于 $z$ 的切口终止。  
因为到该切口的一次切割最长 $b$ ，候选终点只需考虑 $[z, z+b]$ ；之后可恢复目标定长切割，  
直到新的异常通知。所有候选终点也按两级目标比较，完全并列时再依次选更少切口和更早终点。  
有限无环图的递推给出当前已知信息与网格约束下的精确最优方案。"),

text("每次通知先执行上一方案中启动时间严格早于本次通知的切口，  
将这些切口永久保留；再加入新报废区间，只重算剩余部分。同一异常序列的任意前缀，  
必须与在更长序列中运行到该时刻的决策一致。全部异常仅用于最后的事后下界计算，  
不传入尚未到达的在线决策。"),

```

text(f"问题二累计实际报废为{losses['Q2']:.1f}米。以下表中的方案均从
“已锁定”切口向后依次列一次切割长度，顿号分隔、乘号合并连续等长件；“余下损失”
是该时刻已知缺陷下的新待执行方案损失，不是当次新增损失，不能按行相加。下一刀“不变”
表示最近的尚未启动切口不调整，其后的切口仍可重排。第一行给出首次方案，
后续行保留修改前后的待切方案。")],2,key="q2")
def event_tables(key,number,title,offset=0):
    rows=data["online"][key]["events"]
    for first in range(0,9,3):
        section(f"{number}.{offset+first//3+1} "+title+f":
第{first+1}—{first+3}次通知",[
            table(["通知/min;
下一刀","已锁定/m","原待切长度/m","新待切长度/m","余下损失/m"])+[
                f"{r['time_tick']/10:.1f}; "+("首次" if
r["current_next_cut_changed"] is None else "调整" if
r["current_next_cut_changed"] else "不变"),f"{r['locked_boundary']/10:.1f
}",plan_text(r["previous_pending"]),plan_text(r["new_plan"]["pieces"]),f
"{r['new_plan']['score'][0]/10:.1f}" for r in
rows[first:first+3]))],3)
event_tables("Q2","5.2"," 问题二事件方案")
section("5.3 问题三：替换产品区间后的决策",[
    text(f"问题三两组累计实际报废分别为{losses['Q3a']:.1f}米和{losses['Q3b
']:.1f}米。沿用同一状态、回收定义和信息边界，只替换l、u、d，分别得到下列方案。
每块一次切割件的绝对起止位置及离线回收区间另由event_plans.csv完整列出，
可逐行核对正文长度表。"),
    text("在266.0与270.7分钟的两次异常之间，健康材料仅长4.7-0.8=3.9米，
三种目标都不能回收。第二次通知时，这两段缺陷还在上游，相关切口尚可调整。
图1以11.1米目标为例展示局部修改；橙色的新报废段在旧方案形成时尚未知，
旧方案不是已经执行的错误切割。"),
    {"image":"artifacts/figures/online_adjustment.pdf","caption":"图1
相邻异常附近的待切方案调整。黑色短线为一次切口，蓝色为可回收区间，灰色为舍弃部分；
两条均为局部截取。"}],2,key="q3")
event_tables("Q3a","5.3"," 目标 8.5 米")
event_tables("Q3b","5.3"," 目标 11.1 米",offset=3)
section(" 六、验证与灵敏度分析",[
    section("6.1 独立网络流复算与连续下界",[
        text("独立程序先把材料离散为健康/报废单元，逐单元扫描每条边中的最长健康连段，
未调用主程序的区间回收函数。以单位流守恒构造最小费用流线性规划，先最小化损失，
再固定最小损失求最小偏差，使用HiGHS求解。无环图的网络流可行域具有整数顶点；
程序同时核对流量整数性与守恒残差。12种尾坯损失，以及27次在线决策的损失与偏差，
均与主算法一致。尾坯连续偏差另按有理数恒等式核对，未将网格偏差冒充连续偏差。"),

```

text(r"为判断总损失是否仍受网格限制，事后使用完整已实现异常序列构造连续下界。对长度 $\ell_j$ 的有限健康间隔，最多形成 $\lfloor \ell_j / l \rfloor$ 个合格品，总回收量不超过 $\min\{\ell_j, u \lfloor \ell_j / l \rfloor\}$ 。再加上报废段本身，得"),

$$eq(r"W_{\mathrm{LB}} = \sum_j |D_j| + \sum_j \left[ \ell_j - \min\left\{ \ell_j, u \left\lfloor \ell_j \frac{1}{l} \right\rfloor \right\} \right] \right),$$

text("该下界放宽了运输、机器周期和在线限制，因此即使允许连续切口并提前知道全部异常，也不可能低于它。初始健康区间从声明的初始切口开始计算；最后报废段后的无限健康延续无需额外损失。"),

| 目标/m                                    | 在线实际损失/m      | 连续下界/m                                                       | 是否达到                                                                  |
|-----------------------------------------|---------------|--------------------------------------------------------------|-----------------------------------------------------------------------|
| data[online][k]['case']['target_m']:.1f | losses[k]:.1f | verified[online][k]['continuous_lower_bound']['waste_m']:.1f | 是 if verified[online][k]['continuous_lower_bound']['attained'] else 否 |

for k in losses)),

text("三组损失均达到下界，故第一目标在本初始状态和给定序列下已经最优。这不能推出该策略对任何未来序列均具有离线最优表现，也不能推出整个已实现序列的二级偏差全局最优；后者只在每次已知信息与当前网格的规划问题中得到最优性验证。")),2)

section("6.2 基线、网格与初始相位",[

text("固定目标长度切割作为基线，其切口完全不依据异常调整，仅在切下后尽量离线回收。它与在线策略均处理到最后报废段后的一个完整切口；多出的健康目标定长件不增加损失。基线与本文方案的差别反映调整切口的价值，不是换一个算法名称。"),

| 目标/m                                    | 定长基线损失/m                               | 在线损失/m                   | 0.1米偏差/m <sup>2</sup>         | 0.05米偏差/m <sup>2</sup>             |
|-----------------------------------------|----------------------------------------|--------------------------|-------------------------------|------------------------------------|
| data[online][k]['case']['target_m']:.1f | s['fixed_target_baseline_waste_m']:.1f | s['nominal_waste_m']:.1f | s['nominal_deviation_m2']:.3f | s['fine_grid']['deviation_m2']:.3f |

for k,s in sensitivity['cases'].items())),

text("网格加密至0.05米后，三种累计损失均不变，但长度偏差有所下降。这与损失已达到连续下界一致，也说明二级偏差仍有离散影响。将初始已启动切口移至负半个目标长度，所得损失如下："),

| 目标/m                                    | 初始切口/m                                                   | 变更后损失/m                                       | 该状态下的下界/m                                                    |
|-----------------------------------------|----------------------------------------------------------|-----------------------------------------------|--------------------------------------------------------------|
| data[online][k]['case']['target_m']:.1f | s['half_target_initial_phase']['initial_boundary_m']:.2f | s['half_target_initial_phase']['waste_m']:.2f | s['half_target_initial_phase']['lower_bound']['waste_m']:.2f |

for k,s in sensitivity['cases'].items())),

text(f"初始相位改变了第一段可用健康材料的长度，不能把不同状态下的损失当成同一问题的相互矛盾答案。原题未给该状态，因此实际部署应读取最近已启动切口，而不是固定套用本文数值。0.1米网格下27次规划的本机最长耗时为{max\_seconds:.3f}秒；计时不含事后独立LP验证，不构成设备实时性的硬保证。")),2)

section(" 七、模型评价与改进",[

```

        text("模型利用一次运输件最多容纳一个合格产品的结构，
避免多件装箱的额外组合变量；用字典序比较直接表达题目优先级，
避免任意加权把少报废与接近目标混为一谈。在线执行记录保存已启动切口与每次通知前后的方案，
能够检验是否偷用了未来信息。独立网络流与连续下界分别核对网格两级目标和连续总损失。"),
        text("主要边界是初始切口、缺陷中心对齐、零切缝和离线能力无限。
实际切缝会改变每刀的质量守恒，离线容量可能使即时损失最小方案难以执行；
需接入实际设备状态再规划。当前次优先级取平方偏差，其他用户偏好需重新求解。
本文未给出所有可能异常序列下的竞争比，也未证明二级目标的连续最优性。"))
    section(" 八、结论",[
        text(f"在声明的初始状态及加工假设下，
本文完成12种尾坯和三种目标下各9次异常决策。9.5、8.5、
11.1米目标的在线累计损失分别为{losses['Q2']:.1f}、{losses['Q3a']:.1f}、
{losses['Q3b']:.1f}米，均达到相应连续材料下界。方程、
逐次方案与独立计算共同支撑这些条件结论；对初始相位与次优先级的边界已经明确。"))
    section("AI工具使用声明",[text("本训练稿使用AI工具参与题意分析、模型推导、
代码编写、验证和文字排版。尚未完成参赛团队的人工审核、正式AI使用详情与比赛提交材料审查，
不能作为已经获准提交的论文。"))
    section("参考文献",[text("[1] 全国大学生数学建模竞赛组委会。
2021年高教社杯全国大学生数学建模竞赛D题：连铸切割的在线优化[Z]。2021。原题两页及图1、
图2；使用本项目冻结的原始PDF。"))
    sections.append(support_appendix(run, support_root))
    output=output/"paper";output.mkdir(exist_ok=True)
    write_json(output/"document.json",{"title":"连铸切割的在线优化与最优性验证",
"sections":sections})
    print({"sections":len(sections),"paragraph_characters":sum(len(b.get
("text","")) for s in sections for b in
s["blocks"])}),flush=True)

```

```

if __name__=="__main__":
    parser=argparse.ArgumentParser(description=__doc__);parser.add_argum
ent("run",type=Path)
    compose(parser.parse_args()).run()

```

plots.py

```

"""Plot numerical cut intervals and defect locations with Chinese
labels."""
import argparse
import os
from pathlib import Path
from solve import read_json

```

```

def draw(run):
    cache=run/"tmp/matplotlib"; cache.mkdir(parents=True,exist_ok=True)
    os.environ["MPLCONFIGDIR"]=str(cache.resolve())
    os.environ["XDG_CACHE_HOME"]=str((run/"tmp").resolve())
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from matplotlib import font_manager
    from matplotlib.patches import Patch
    songti=Path("/System/Library/Fonts/Supplemental/Songti.ttc")
    if songti.exists():
        from fontTools.ttLib import TTCollection
        fonts=TTCollection(songti); path=cache/"SongtiSC-Regular.ttf"
        fonts.fonts[6].save(path); fonts.close()
        font_manager.fontManager.addfont(str(path))
        family=font_manager.FontProperties(fname=str(path)).get_name()
    else:
        available={f.name for f in font_manager.fontManager.ttflist}
        family=next((f for f in ("SimSun","Noto Serif CJK SC","Source
Han Serif SC") if f in available),None)
        if not family: raise RuntimeError("A Chinese serif font is
needed for the paper figures")
        plt.rcParams.update({"font.family":family,"font.size":11,"axes.unico
de_minus":False,

"axes.spines.top":False,"axes.spines.right":False,"pdf.fonttype":42})
        result=read_json(run/"artifacts/solution.json")
        row=result["online"]["Q3b"]["events"][7]
        fig,ax=plt.subplots(figsize=(8.1,3.4))
        for y,edges in
[(1,row["previous_pending"]),(0,row["new_plan"]["pieces"]]):
            for piece in edges:
                a,b=piece["start"]/10,piece["end"]/10
                if b<310 or a>344: continue
                ax.broken_barh([(a,b-a)],(y-.16,.32),facecolors="#d7dce0",ed
gecolors="white",linewidth=1)
                if piece["recovered"]:
                    left,right=[x/10 for x in piece["recovered"]]

ax.broken_barh([(left,right-left)],(y-.16,.32),facecolors="#0072B2")

```



```

        if 311<a<343:
ax.vlines(a,y-.22,y+.22,color="#303030",linewidth=1)
        if 311<b<343:
ax.vlines(b,y-.22,y+.22,color="#303030",linewidth=1)
        for a,b in [(325.6,326.4),(330.3,331.1)]:
            ax.axvspan(a,b,color="#D55E00",alpha=.35)
            ax.annotate("旧异常",(326,1.2),xytext=(322.2,1.55),arrowprops={"arrow
style":"-", "color":"#555"})
            ax.annotate("270.7分时新获知",(330.7,1.2),xytext=(330.7,1.55),arrowpro
ps={"arrowstyle":"-", "color":"#555"},ha="center")
            ax.set(xlim=(310,344),ylim=(-.65,1.95),yticks=[0,1],yticklabels=["新
通知后的方案","通知前的待执行方案"],
                xlabel=" 材料坐标 / m (亦为到达切割起点的时刻 / min) ")
            ax.legend(handles=[Patch(facecolor="#0072B2",label="可回收产品"),Patch
(facecolor="#d7dce0",label="待舍弃部分"),
Patch(facecolor="#D55E00",alpha=.35,label="报废段位置")],loc="lower
center",ncol=3,frameon=False,fontsize=10)
            ax.grid(axis="x",alpha=.15); fig.tight_layout()
            out=run/"artifacts/figures";out.mkdir(exist_ok=True)
            fig.savefig(out/"online_adjustment.pdf",bbox_inches="tight")
            fig.savefig(out/"online_adjustment.png",dpi=300,bbox_inches="tight")
            plt.close(fig)

if __name__=="__main__":
    parser=argparse.ArgumentParser(description=__doc__);
    parser.add_argument("run",type=Path)
    draw(parser.parse_args().run)

```

render\_latex.py

```

"""Typeset Chinese papers with real inline/display mathematics using
XeTeX/Tectonic."""
import argparse
import hashlib
import json
import os
import platform
import re
import shutil
import subprocess

```

```

from pathlib import Path

def escape_text(text):
    replacements = {"\\": r"\textbackslash{}", "&": r"\&", "%": r"\%",
"$": r"\$", "#": r"\#",
        "_": r"\_", "{": r"\{", "(": r"\(", "~":
r"\textasciitilde{}", "^": r"\textasciicircum{}"}
    return "".join(replacements.get(character, character) for character
in str(text))

def inline(text):
    text = str(text)
    if text.count("$") % 2:
        raise ValueError("unbalanced inline math delimiters")
    return "".join(piece if piece.startswith("$") and
piece.endswith("$") else plain_with_units(piece)
        for piece in re.split(r"(\${^$}*\$)", text))

def plain_with_units(text):
    units = {"cm · min-1": r"${\mathrm{cm}}\,\,\mathrm{min}^{\{-1\}}$", "° · s":
r"${}^{\circ}\mathrm{C}\,\,\mathrm{s}$",
        "°/s": r"${}^{\circ}\mathrm{C}/\mathrm{s}$", "°":
r"${}^{\circ}\mathrm{C}$"}
    pattern = "(" + "|".join(re.escape(unit) for unit in units) + ")"
    return "".join(units.get(piece, escape_text(piece)) for piece in
re.split(pattern, text))

def render_latex(run, source, *, compiler=None, cache_dir=None,
compile_pdf=True):
    run = Path(run).resolve()
    source = Path(source).resolve()
    if (run / "state.json").exists() and json.loads((run /
"state.json").read_text())["stage"] == "VERIFIED":
        raise ValueError("create a new version before typesetting a
sealed run")
    data = json.loads(source.read_text(encoding="utf-8"))
    output = run / "paper"
    output.mkdir(exist_ok=True)

```

```

    document_sha256 = hashlib.sha256(source.read_bytes()).hexdigest()
    (output /
"render_report.json").write_text(json.dumps({"render_status": "PENDING",
"source": source.name,

"document_sha256": document_sha256,

"note": "An
existing PDF may be from an earlier generation; not accepted until
compilation succeeds."},

ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    if platform.system() == "Darwin":
        fonts = r"\setmainfont{Times New Roman}\setCJKmainfont{Songti
SC}\setCJKsansfont{Heiti SC}\setCJKmonofont{Songti SC}"
    else:
        fonts = r"\setmainfont{TeX Gyre Termes}\setCJKmainfont{FandolSon
g-Regular.otf}[BoldFont=FandolSong-Bold.otf]\setCJKsansfont{FandolHei-Re
gular.otf}\setCJKmonofont{FandolSong-Regular.otf}"
        has_code = any("code" in block for section in data["sections"] for
block in section["blocks"])
        if has_code:
            fonts += r"\usepackage{fvextra}"
            if platform.system() == "Darwin":
                fonts += r"\setmonofont{Menlo}"
            else:
                fonts += r"\IfFontExistsTF{DejaVu Sans
Mono}{\setmonofont{DejaVu Sans Mono}}{}"
                # Celsius is present in Chinese fonts but absent from
                # Menlo/DejaVu Sans Mono.
                # Preserve the literal source character in code instead of
                # replacing it.
                fonts += r'\xeCJKDeclareCharClass{CJK}{\2103}'
            code_receipts = []
            preamble = r"""\documentclass[12pt,a4paper]{article}
\usepackage{fontspec,xecjk,amsmath,amssymb,geometry,graphicx,booktabs,ta
bularx,array}
\usepackage{titlesec,caption,placeins,indentfirst,float}
\geometry{margin=28mm,footskip=12mm}
\begin{document}
\XeTeXlinebreaklocale "zh"
\XeTeXlinebreakskip=0pt plus 1pt
\setlength{\parindent}{2em}

```

```

\setlength{\parskip}{0.25em}
\linespread{1.35}
\titleformat{\section}{\sffamily\bfseries\large}{}{0em}{}
\titleformat{\subsection}{\sffamily\bfseries\normalsize}{}{0em}{}
\titleformat{\subsubsection}{\sffamily\normalsize}{}{0em}{}
\titlespacing*{\section}{0pt}{1.2em}{0.7em}
\captionsetup{font=small,skip=6pt}
\renewcommand{\arraystretch}{1.35}
\setlength{\emergencystretch}{2em}
\begin{document}
"""".replace("FONTS", fonts)
    lines = [preamble, r"\begin{center}{\sffamily\bfseries\LARGE " +
inline(data["title"]) + r"}\end{center}"]
    markdown = ["# " + data["title"], ""]
    if data.get("status_note"):
        lines.append(r"\begin{center}\small " +
inline(data["status_note"]) + r"\end{center}")
        markdown.extend(["> " + data["status_note"], ""])
    for section in data["sections"]:
        lines.append(r"\FloatBarrier")
        if section.get("page_break_before"):
            lines.append(r"\clearpage")
        level = section.get("level", 1)
        if level not in (1, 2, 3):
            raise ValueError("section level must be 1, 2 or 3")
        command = {1: "section", 2: "subsection", 3:
"subsubsection"}[level]
        lines.append("\\ " + command + "*{" + inline(section["title"]) +
"}")
        markdown.extend(["#" * (level + 1) + " " + section["title"],
""])
        for block in section["blocks"]:
            if "text" in block:
                lead = block.get("lead", "")
                prefix = r"{\sffamily\bfseries " + inline(lead) + "}" if
lead else ""
                lines.extend([prefix + inline(block["text"]), r"\par"])
                markdown.extend(["**" + lead + "** " if lead else ""] +
block["text"], ""])
            elif "equation" in block:
                formula = block["equation"]

```

```

        lines.extend([r"\begin{equation}", formula,
r"\end{equation}"])
        markdown.extend(["$$", formula, "$$", ""])
    elif "code" in block:
        code = block["code"]
        if not isinstance(code, str) or not code.strip():
            raise ValueError("source-code blocks require
nonempty text")
        name = block.get("filename", "source")
        code_file = output /
f"source-{len(code_receipts)+1:03d}.txt"
        code_file.write_text(code, encoding="utf-8")
        code_receipts.append({"filename": name, "utf8_sha256":
hashlib.sha256(code.encode("utf-8")).hexdigest()})
        lines.extend([r"\subsubsection*{" + escape_text(name) +
}],

r"\VerbatimInput[fontsize=\footnotesize,breaklines=true,breakanywhere=tr
ue,breakautoindent=false,breakindent=0pt,breaksymbolleft={},breaksymbolr
ight={},breakanywheresymbolpre={},breakanywheresymbolpost={}]{ " +
code_file.name + "}")

        longest = max((len(match.group()) for match in
re.finditer(r"``+", code)), default=0)
        fence = "``" * max(3, longest+1)
        markdown.extend(["### " + name, "", fence,
code.rstrip("\n"), fence, ""])
    elif "table" in block:
        rows = block["table"]
        columns = len(rows[0])
        if any(len(row) != columns for row in rows):
            raise ValueError("ragged paper table")
        lines.extend([r"\begin{table}[H]\centering\small",
r"\begin{tabularx}{\linewidth}{ " +
r">{\raggedright\arraybackslash}X" * columns + "}", r"\toprule"])
        for index, row in enumerate(rows):
            formatted = []
            for cell in row:
                if index == 0 and "/" in str(cell) and "$" not
in str(cell):
                    label, unit = str(cell).split("/", 1)
                    formatted.append(r"\shortstack[l]{ " +
inline(label) + r"\\(" + inline(unit) + ")}")

```

```

        else:
            formatted.append(inline(cell))
        lines.append(" & ".join(formatted) + r" \\")
        if index == 0:
            lines.append(r"\midrule")
        markdown.append("| " + " | ".join(str(cell) for cell
in row) + " |")

        if index == 0:
            markdown.append("| " + " | ".join("----" for _ in
row) + " |")

        lines.extend([r"\bottomrule\end{tabularx}\end{table}"])
        markdown.append("")
    elif "image" in block:
        image = (run / block["image"]).resolve()
        image.relative_to(run)
        relative = os.path.relpath(image,
output).replace(os.sep, "/")
        if any(character in relative for character in
"{}%\n\r"):
            raise ValueError("unsupported characters in figure
path")

        lines.extend([r"\begin{figure}[H]\centering",

r"\includegraphics[width=\linewidth]{\detokenize{" + relative + "}}"])
        if block.get("caption"):
            lines.append(r"\caption*{" +
inline(block["caption"]) + "}")
            lines.append(r"\end{figure}")
            markdown.extend([f"![{block.get('caption',
'')}]({relative})", ""])
        else:
            raise ValueError("unknown paper block")
        lines.append(r"\end{document}")
        tex = output / "main.tex"
        tex.write_text("\n".join(lines) + "\n", encoding="utf-8")
        (output / "main.md").write_text("\n".join(markdown),
encoding="utf-8")
        if compile_pdf:
            executable = compiler or shutil.which("tectonic") or
shutil.which("xelatex")
            if not executable:

```

```

        raise ValueError("TeX source written; install
Tectonic/XeLaTeX or supply --compiler. Do not fall back to plaintext
formulas.")
    env = os.environ.copy()
    env["TECTONIC_CACHE_DIR"] = str(Path(cache_dir).resolve() if
cache_dir else run / "tmp/tectonic-cache")
    if "tectonic" in Path(executable).name.lower():
        command = [str(executable), "-X", "compile", "--untrusted",
"--keep-logs", "--outdir", str(output), str(tex)]
    else:
        command = [str(executable), "-no-shell-escape",
"-interaction=nonstopmode", "-halt-on-error", "-output-directory",
str(output), str(tex)]
    completed = subprocess.run(command, cwd=output, env=env,
capture_output=True, text=True)
    (output / "typesetting.log").write_text(completed.stdout +
completed.stderr, encoding="utf-8")
    if completed.returncode:
        raise RuntimeError("TeX compilation failed; inspect
paper/typesetting.log")
    log = (output / "main.log").read_text(encoding="utf-8",
errors="replace") if (output / "main.log").is_file() else
completed.stdout + completed.stderr
    if "Missing character:" in log:
        raise RuntimeError("TeX reported missing glyphs; inspect
paper/main.log")
    from pypdf import PdfReader
    pdf = PdfReader(output / "main.pdf")
    font_names = sorted({str(font.get_object().get("/BaseFont", ""))
for page in pdf.pages
                        for font in page["/Resources"].get("/Font",
{}).values()})
    report = {"pages": len(pdf.pages), "encrypted":
pdf.is_encrypted,
            "text_characters": sum(len(page.extract_text() or "")
for page in pdf.pages),
            "visual_review": "PENDING", "source": source.name,
"backend": "latex",
            "fonts": font_names, "pdf_sha256":
hashlib.sha256((output / "main.pdf").read_bytes()).hexdigest(),
            "document_sha256": document_sha256, "render_status":
"COMPILED", "code_blocks": code_receipts}

```

```

        (output / "render_report.json").write_text(json.dumps(report,
ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
        print({"tex": str(tex), "pdf_compiled": compile_pdf,
"visual_review": "PENDING"})

```

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--run", type=Path, required=True)
    parser.add_argument("--source", type=Path, required=True)
    parser.add_argument("--compiler")
    parser.add_argument("--cache-dir", type=Path)
    parser.add_argument("--source-only", action="store_true")
    args = parser.parse_args()
    render_latex(args.run, args.source, compiler=args.compiler,
cache_dir=args.cache_dir, compile_pdf=not args.source_only)

```

reproduce.py

```

"""Portable support-package entry point. Run with Python -I in a new
output directory."""

```

```

import argparse
import copy
import json
import os
from pathlib import Path, PurePosixPath
import subprocess
import sys

```

```

# -I deliberately excludes cwd/PYTHONPATH; allow only this extracted
package's code.

```

```

sys.path.insert(0, str(Path(__file__).resolve().parent))
from artifact_io import read_json, write_json, sha256_file

```

```

def comparable_solution(solution):
    value = copy.deepcopy(solution)
    for record in value["online"].values():
        for event in record["events"]:
            event.pop("planning_seconds", None)
    return value

```



```

def comparable_sensitivity(sensitivity):
    value = copy.deepcopy(sensitivity)
    value.pop("solution_sha256", None)
    return value

def check_manifest(root):
    manifest = read_json(root / "manifest.json")
    seen = set()
    for item in manifest["files"]:
        name = item["path"]
        if (not isinstance(name, str) or "\\" in name or ":" in name or
            PurePosixPath(name).is_absolute() or ".." in
PurePosixPath(name).parts or name in seen):
            raise ValueError("nonportable or duplicate manifest path")
        seen.add(name)
        path = root / name
        path.resolve().relative_to(root.resolve())
        if not path.is_file() or path.is_symlink() or sha256_file(path)
!= item["sha256"]:
            raise ValueError("package file missing or changed: " + name)
        required = {"artifact_io.py", "solve.py", "verify.py",
"sensitivity.py", "reproduce.py",
                    "facts.json", "expected-results.json",
"requirements.txt"}
        if not required <= seen:
            raise ValueError("support manifest omits required files")
    return manifest

def reproduce(root, output, source=None):
    if not __debug__:
        raise ValueError("Verification requires assertions; run without
-0")
    root, output = Path(root).resolve(), Path(output).resolve()
    check_manifest(root)
    facts = read_json(root / "facts.json")
    if source is not None and sha256_file(source) !=
facts["source_sha256"]:
        raise ValueError("supplied original PDF does not match the
reviewed source")

```

```

output.mkdir(parents=True, exist_ok=False)
write_json(output / "artifacts/facts.json", facts)
from solve import solve
from verify import verify
from sensitivity import assess
solve(output)
verification = verify(output)
assess(output)
# Expected records are opened only after all computations have
completed.
expected = read_json(root / "expected-results.json")
actual = read_json(output / "artifacts/solution.json")
sensitivity = read_json(output / "artifacts/sensitivity.json")
checks = {
    "all_tail_and_online_decisions": comparable_solution(actual) ==
expected["solution"],
    "baseline_grid_and_phase_sensitivity":
comparable_sensitivity(sensitivity) == expected["sensitivity"],
    "independent_network_and_interval_verification":
verification["passed"] is True,
}
report = {"passed": all(checks.values()), "checks": checks,
        "source_pdf_checked": source is not None,
        "input_scope": "reviewed statement parameters in
facts.json; optional original PDF checked by hash, not reinterpreted
automatically",
        "comparison_scope": "all deterministic solution fields
including cuts, recovered intervals and objectives; only
planning_seconds excluded; sensitivity source hash excluded because
timings change solution hash",
        "manifest_sha256": sha256_file(root / "manifest.json"),
        "solution_sha256": sha256_file(output /
"artifacts/solution.json"),
        "submission_ready": False}
write_json(output / "reproduction_report.json", report)
if not report["passed"]:
    raise ValueError("reproduced results differ from the packaged
reference")
return report

def main():

```

```

parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("--output", type=Path, required=True)
parser.add_argument("--source", type=Path, help="optional original
PDF; hash checked against the reviewed source")
parser.add_argument("--paper", action="store_true", help="also
regenerate figures, full source appendix and PDF")
parser.add_argument("--paper-python", default=sys.executable)
parser.add_argument("--latex-compiler")
parser.add_argument("--cache-dir", type=Path)
args = parser.parse_args()
root = Path(__file__).resolve().parent
output = args.output.resolve()
report = reproduce(root, output, args.source)
if args.paper:
    report["paper"] = {"render_status": "PENDING", "visual_review":
"PENDING"}
    write_json(output / "reproduction_report.json", report)
    from plots import draw
    from paper import compose
    draw(output)
    compose(output, support_root=root)
    command = [args.paper_python, "-I", str(root /
"render_latex.py"), "--run", str(output),
                "--source", str(output / "paper/document.json")]
    if args.latex_compiler:
        command.extend(["--compiler", args.latex_compiler])
    if args.cache_dir:
        command.extend(["--cache-dir",
str(args.cache_dir.resolve())])
    environment = os.environ.copy()
    environment.pop("PYTHONPATH", None)
    subprocess.run(command, check=True, env=environment)
    report["paper"] = {"render_status": "COMPILED", "pdf_sha256":
sha256_file(output / "paper/main.pdf"), "visual_review": "PENDING"}
    write_json(output / "reproduction_report.json", report)
    print(json.dumps(report, ensure_ascii=False, indent=2), flush=True)

if __name__ == "__main__":
    main()

```

sensitivity.py

```
"""Grid refinement, initial phase, and a fixed-target cutting
baseline."""
import argparse
from pathlib import Path
from solve import online, recover
from verify import verify_trace, continuous_lower_bound
from artifact_io import read_json, write_json, sha256_file

def assess(run):
    facts=read_json(run/"artifacts/facts.json")
    primary=read_json(run/"artifacts/solution.json")
    records={}
    for name,case in facts["cases"].items():
        events=facts["event_times_min"]
        fine=online(events,case,scale=20)
        fine_trace=verify_trace(fine,events)
        d,lo,hi=[round(case[k]*10) for k in
("target_m","lower_m","upper_m")]
        defects=[(round(t*10)+596,round(t*10)+604) for t in events]
        anchor=0; baseline=[]
        while anchor<defects[-1][1]:
            # Cut positions are completely independent of defects;
offline recovery occurs afterward.
            baseline.append(recover(anchor,anchor+d,defects,lo,hi,d))
            anchor+=d
            shifted=online(events,case,scale=20,initial_boundary=-d)
            shifted_trace=verify_trace(shifted,events)
            shifted_bound=continuous_lower_bound(events,case["lower_m"],case
["upper_m"],initial=-d/20)
            records[name]={"nominal_waste_m":primary["online"][name]["total_
waste_m"],
                        "fixed_target_baseline_waste_m":sum(e["waste"]
for e in baseline)/10,

"fine_grid":{"step_m":.05,"waste_m":fine["total_waste_m"],

"deviation_m2":fine["total_deviation_m2"],"trace":fine_trace},

"half_target_initial_phase":{"initial_boundary_m":-d/20,
```

```

        "waste_m":shifted["total_waste_m"],"
trace":shifted_trace,"lower_bound":shifted_bound},

"nominal_deviation_m2":primary["online"][name]["total_deviation_m2"]}
    print(name,records[name],flush=True)
    output={"solution_sha256":sha256_file(run/"artifacts/solution.json")
,"cases":records,
        "scope":"grid refinement and one initial-phase alternative;
no uniform guarantee over all phases or events"}
    write_json(run/"artifacts/sensitivity.json",output)

if __name__=="__main__":
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument("run",type=Path)
    assess(parser.parse_args()).run()

```

solve.py

```

"""Exact continuous tail enumeration and causal grid-optimal primary-cut
planning."""
import argparse
import csv
from fractions import Fraction
from pathlib import Path
from time import perf_counter

from artifact_io import read_json, write_json, sha256_file

def exact_tail(length, target=9.5, lower=9., upper=10.):
    total, d, lo, hi = map(lambda x: Fraction(str(x)),
(length,target,lower,upper))
    a, b = Fraction("4.8"), Fraction("12.6")
    best = None
    for count in range(1, int(total/a)+1):
        for good in range(count+1):
            scrap = count-good
            if not good*lo+scrap*a <= total <= count*b:
                continue
            recovered = min(good*hi, total-scrap*a)
            if recovered < good*lo:

```

```

        continue
    deviation = good*(recovered/good-d)**2 if good else
Fraction(0)
    score = (total-recovered, deviation, count, scrap)
    if best is None or score < best[0]:
        best = score, good, scrap, recovered
    if best is None:
        raise ValueError("tail cannot be partitioned into transportable
primary pieces")
    score, good, scrap, recovered = best
    mean = recovered/good if good else Fraction(0)
    accepted = [mean]*good + [Fraction(0)]*scrap
    primary = [mean]*good + [a]*scrap
    extra = total-sum(primary)
    for i in range(len(primary)):
        addition = min(extra,b-primary[i])
        primary[i] += addition
        extra -= addition
    assert extra == 0 and sum(primary) == total
    return {"tail_m":float(total), "primary_m":list(map(float,primary)),
            "recovered_m":list(map(float,accepted)),
"waste_m":float(score[0]),
            "squared_deviation_m2":float(score[1]),
"accepted_count":good,
            "exact": {"primary":list(map(str,primary)),
"recovered":list(map(str,accepted)),
                    "waste":str(score[0]),
"squared_deviation":str(score[1])}}

```

```

def recover(left, right, defects, lower, upper, target):
    """At most one accepted product fits: primary max 12.6 < 2*8.0 m."""
    cursor = left
    longest = (left,left)
    for begin,end in defects:
        if end <= left or begin >= right:
            continue
        gap = (cursor,min(begin,right))
        if gap[1]-gap[0] > longest[1]-longest[0]:
            longest = gap
        cursor = max(cursor,min(end,right))
    if right-cursor > longest[1]-longest[0]:

```

```

        longest = (cursor, right)
        size = min(upper, longest[1]-longest[0]) if longest[1]-longest[0] >=
lower else 0
        kept = [longest[0], longest[0]+size] if size else None
        return {"start":left, "end":right, "recovered":kept,
                "waste":right-left-size, "deviation":(size-target)**2 if
size else 0}

```

```

def plan(start, notification, defects, target, lower, upper, scale=10):
    """Shortest lexicographic path to the first cut beyond all known
defects."""
    a, b = 48*scale//10, 126*scale//10
    last_bad = max(end for _,end in defects)
    limit = last_bad+b
    cost = {start:(0,0,0)}
    predecessor = {}
    evaluated = 0
    for left in range(start, last_bad):
        if left not in cost:
            continue
        for right in range(max(left+a, notification if left==start else
left+a), min(left+b, limit)+1):
            edge = recover(left, right, defects, lower, upper, target)
            evaluated += 1
            score = (cost[left][0]+edge["waste"],
cost[left][1]+edge["deviation"], cost[left][2]+1)
            if right not in cost or score < cost[right]:
                cost[right] = score
                predecessor[right] = (left, edge)
    terminals = [end for end in cost if end>=last_bad]
    if not terminals:
        raise ValueError("no causal transport-feasible plan")
    finish = min(terminals, key=lambda x: (*cost[x], x))
    edges = []
    cursor = finish
    while cursor != start:
        cursor, edge = predecessor[cursor]
        edges.append(edge)
    edges.reverse()
    return {"start":start, "notification":notification,
"defects":[list(x) for x in defects],

```

```

        "terminal":finish, "score":list(cost[finish]),
"pieces":edges,
        "grid_scale_per_m":scale, "evaluated_edges":evaluated,
        "optimality":"exact lexicographic optimum of the declared
finite grid graph; continuous/global online optimality not implied"}

def online(event_times, case, scale=10, initial_boundary=0):
    if scale not in (10,20):
        raise ValueError("this benchmark supports exact 0.1 or 0.05 m
grids")
    def ticks(x):
        value=Fraction(str(x))*scale
        if value.denominator!=1:
            raise ValueError("input is not exactly representable on the
declared grid")
        return int(value)
    target,lower,upper = [ticks(case[key]) for key in
("target_m","lower_m","upper_m")]
    executed = []
    pending = []
    known = []
    history = []
    anchor = initial_boundary
    for index,time in enumerate(event_times):
        now = ticks(time)
        while pending and pending[0]["end"] < now:
            piece = pending.pop(0)
            executed.append(piece)
            anchor = piece["end"]
        # Healthy continuation after a former finite plan, without
future events.
        while not pending and anchor+target < now:
            edge =
recover(anchor,anchor+target,known,lower,upper,target)
            edge.update(planned_at=history[-1]["time_tick"] if history
else initial_boundary,
                        known_event_count=index)
            executed.append(edge)
            anchor = edge["end"]
        before = [dict(edge) for edge in pending]
        known.append((now+ticks(59.6),now+ticks(60.4)))

```



```

        future = [interval for interval in known if interval[1]>anchor]
        started = perf_counter()
        chosen = plan(anchor,now,future,target,lower,upper,scale)
        elapsed = perf_counter()-started
        pending = [dict(edge,planned_at=now,known_event_count=index+1)
for edge in chosen["pieces"]]

history.append({"event":index+1,"time_tick":now,"known_events":index+1,

"locked_boundary":anchor,"committed_piece_count":len(executed),
               "cut_in_progress":anchor<=now<anchor+3*scale,

"previous_pending":before,"new_plan":chosen,"planning_seconds":elapsed,
               "current_next_cut_changed":None if not before
else before[0]["end"]!=pending[0]["end"],
               "committed_waste":sum(edge["waste"] for edge in
executed),
               "projected_total_waste":sum(edge["waste"] for
edge in executed)+chosen["score"][0]})
    executed.extend(pending)
    return
{"case":case,"scale":scale,"initial_boundary":initial_boundary,
 "events":history,"executed":executed,
 "total_waste_m":sum(edge["waste"] for edge in
executed)/scale,
 "total_deviation_m2":sum(edge["deviation"] for edge in
executed)/scale**2,
 "last_cut_start_min":executed[-1]["end"]/scale if executed
else initial_boundary/scale,
 "last_cut_complete_min":executed[-1]["end"]/scale+3 if
executed else initial_boundary/scale+3,
 "scope":"optimal current-known-defect plans on a 0.1/0.05 m
grid; realized final record follows causal revisions"}

def solve(run):
    facts_path = run/"artifacts/facts.json"
    facts = read_json(facts_path)
    tails = [exact_tail(value) for value in facts["tail_lengths_m"]]
    result = {"facts_sha256":sha256_file(facts_path), "tails":tails,
"online":{}}
    for key,case in facts["cases"].items():

```

```

        result["online"][key] = online(facts["event_times_min"],case)
        print(key, {"waste_m":result["online"][key]["total_waste_m"],
                    "event_seconds":[round(row["planning_seconds"],3)
for row in result["online"][key]["events"]]},flush=True)
        write_json(run/"artifacts/solution.json",result)
        with (run/"artifacts/event_plans.csv").open("w",encoding="utf-8-sig"
,newline="") as
stream:
            writer=csv.writer(stream)
            writer.writerow(["case","notification_min","plan","primary_start
_m","primary_end_m",

"sever_cut_start_min","sever_cut_complete_min","machine_return_min",
                    "retained_start_m","retained_end_m","waste_m"])
            for key,record in result["online"].items():
                for row in record["events"]:
                    for kind,pieces in [("previous",row["previous_pending"])
,("new",row["new_plan"]["pieces"])]):
                        for piece in pieces:
                            kept=piece["recovered"]
                            writer.writerow([key,row["time_tick"]/10,kind,pi
ece["start"]/10,piece["end"]/10,

piece["end"]/10,piece["end"]/10+3,piece["end"]/10+4,
                                    kept[0]/10 if kept else
"",kept[1]/10 if kept else "",piece["waste"]/10])
                            print("tails",[r["tail_m"],r["waste_m"]) for r in
tails],flush=True)

if __name__ == "__main__":
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument("run",type=Path)
    solve(parser.parse_args()).run)

```

verify.py

""""Independent cell-occupancy network LP, interval checks and a continuous waste bound."""

```

import argparse
from fractions import Fraction
from pathlib import Path

```

```

import numpy as np
from scipy.optimize import linprog
from scipy.sparse import coo_matrix, csr_matrix, vstack

from artifact_io import read_json, write_json, sha256_file

def network_optimum(start, notification, defects, target, lower, upper,
scale=10, fixed_end=None):
    """Unit-flow LP on an acyclic graph; edge rewards from a cell scan,
not interval recovery."""
    minimum, maximum = int(4.8*scale+.1), int(12.6*scale+.1)
    boundary = fixed_end if fixed_end is not None else max(e for _,e in
defects)
    end = boundary if fixed_end is not None else boundary+maximum
    occupied = [any(a<=x<b for a,b in defects) for x in
range(start,end)]
    sources, destinations, loss, deviation = [],[],[],[]
    for left in range(start,boundary):
        clean = longest = 0
        for right in range(left+1,min(end,left+maximum)+1):
            clean = 0 if occupied[right-start-1] else clean+1
            longest = max(longest,clean)
            if right-left < minimum or (left==start and
right<notification):
                continue
            kept = min(longest,upper) if longest>=lower else 0
            sources.append(left-start); destinations.append(right-start)
            loss.append(right-left-kept)
            deviation.append((kept-target)**2 if kept else 0)
        sink = end-start+1
        terminals = [boundary] if fixed_end is not None else
range(boundary,end+1)
        for point in terminals:
            sources.append(point-start); destinations.append(sink);
loss.append(0); deviation.append(0)
        columns = np.arange(len(sources))
        matrix =
coo_matrix((np.r_[np.ones(len(sources)),-np.ones(len(sources))],
(np.r_[sources,destinations],np.r_[columns,columns])),

```

```

        shape=(sink+1,len(sources))).tocsr()
    balance = np.zeros(sink+1); balance[0]=1; balance[sink]=-1
    first =
linprog(loss,A_eq=matrix,b_eq=balance,bounds=(0,None),method="highs")
    if not first.success:
        raise ValueError("independent network cannot route material:
"+first.message)
    optimum = round(first.fun)
    if abs(first.fun-optimum)>1e-6:
        raise ValueError("nonintegral first objective")
    second = linprog(deviation,A_eq=vstack([matrix,csr_matrix([loss]))),

b_eq=np.r_[balance,optimum],bounds=(0,None),method="highs")
    if not second.success or
np.max(np.abs(second.x-np.round(second.x)))>1e-6:
        raise ValueError("independent lexicographic network result is
not integral")
    return {"loss_tick":optimum,"deviation_tick2":round(second.fun),
        "edges":len(sources),"nodes":sink+1,

"flow_residual":float(np.max(np.abs(matrix@second.x-balance))),
        "method":"two-stage minimum-cost unit-flow linear program,
HiGHS; cell occupancy defines edge recovery"}

def continuous_lower_bound(events, lower, upper, initial=0):
    """Relax transport/cut/causality limits; acceptable products cannot
span a defect."""
    f = lambda x:Fraction(str(x))
    intervals = [(f(t)+f(59.6),f(t)+f(60.4)) for t in events]
    merged = []
    for a,b in intervals:
        if merged and a<=merged[-1][1]:
            merged[-1] = (merged[-1][0],max(b,merged[-1][1]))
        else:
            merged.append((a,b))
    cursor = f(initial)
    waste = Fraction(0)
    gaps = []
    for a,b in merged:
        if b<=cursor:
            continue

```

```

    if a>cursor:
        length = a-cursor
        count = int(length/f(lower))
        recovered_upper = min(length,count*f(upper))
        gaps.append({"length_m":float(length),"max_products":count,"
unavoidable_waste_m":float(length-recovered_upper)})
        waste += length-recovered_upper
        waste += b-max(a,cursor)
        cursor = b
    return
{"waste_m":float(waste),"exact_waste":str(waste),"healthy_gaps":gaps,
    "scope":"continuous material bound using the complete
realized event sequence, only for retrospective verification; not input
to online decisions"}

```

```

def verify_trace(record, event_times):
    scale = record["scale"]
    f = lambda x:int(Fraction(str(x))*scale)
    d,lo,hi = [f(record["case"][k]) for k in
("target_m","lower_m","upper_m")]
    all_bad = [(f(t)+f(59.6),f(t)+f(60.4)) for t in event_times]
    cursor = record["initial_boundary"]
    loss = deviation = 0
    for edge in record["executed"]:
        assert edge["start"]==cursor
        assert f(4.8)<=edge["end"]-cursor<=f(12.6)
        assert edge["end"]-cursor>=f(4) # Cutting + return cycle, in
saw-arrival coordinates.
        assert edge["end"]>=edge["planned_at"]
        kept = edge["recovered"]
        recovered = 0
        if kept is not None:
            a,b=kept; recovered=b-a
            assert cursor<=a<b<=edge["end"] and lo<=recovered<=hi
            assert not any(max(a,x)<min(b,y) for x,y in all_bad)
            expected_waste = edge["end"]-cursor-recovered
            expected_deviation = (recovered-d)**2 if recovered else 0
            assert edge["waste"]==expected_waste and
edge["deviation"]==expected_deviation
            loss += expected_waste; deviation += expected_deviation
            cursor=edge["end"]

```

```

assert loss/scale==record["total_waste_m"]
assert deviation/scale**2==record["total_deviation_m2"]
assert cursor>=max(b for _,b in all_bad)
# Reconstruct which prior plan decisions became irrevocable before
each notification.
for i,row in enumerate(record["events"]):
    now=f(event_times[i])
    committed=[e for e in record["executed"] if e["end"]<now]
    assert row["committed_piece_count"]==len(committed)
    assert row["locked_boundary"]== (committed[-1]["end"] if
committed else record["initial_boundary"])
    assert row["known_events"]==i+1
    expected=[list(x) for x in all_bad[:i+1] if
x[1]>row["locked_boundary"]]
    assert row["new_plan"]["defects"]==expected
    assert all(e["end"]>=now for e in row["new_plan"]["pieces"])
    cursor=row["locked_boundary"]; planned_loss=planned_deviation=0
    for piece in row["new_plan"]["pieces"]:
        assert piece["start"]==cursor and
f(4.8)<=piece["end"]-cursor<=f(12.6)
        kept=piece["recovered"]; size=0
        if kept:
            a,b=kept;size=b-a
            assert cursor<=a<b<=piece["end"] and lo<=size<=hi
            assert not any(max(a,x)<min(b,y) for x,y in expected)
            assert piece["waste"]==piece["end"]-cursor-size
            assert piece["deviation"]==((size-d)**2 if size else 0)
            planned_loss+=piece["waste"];planned_deviation+=piece["devia
tion"];cursor=piece["end"]
        assert row["new_plan"]["score"]==[planned_loss,planned_deviation
,len(row["new_plan"]["pieces"])]
        assert row["new_plan"]["terminal"]==cursor>=max(b for _,b in
expected)
        if i:
            previous=record["events"][i-1]["new_plan"]["pieces"]
            fixed=[e for e in previous if e["end"]<now]
            actual=[e for e in committed if
e["start"]>=record["events"][i-1]["locked_boundary"]]
            assert [(e["start"],e["end"],e["recovered"]) for e in
fixed]==[(e["start"],e["end"],e["recovered"]) for e in
actual[:len(fixed)]]

```

```

    return {"passed":True,"pieces":len(record["executed"]), "total_waste_
m":loss/scale,
           "checks":["mass balance","primary transport
bounds","cut/return spacing","defect-free recovery","target bounds","no
retroactive cut","known-event prefix","irrevocable prior cuts"]}

def verify(run):
    if not __debug__:
        raise ValueError("Verification requires Python assertions; do
not use -O or PYTHONOPTIMIZE")
    facts=read_json(run/"artifacts/facts.json")
    solution=read_json(run/"artifacts/solution.json")
    if
solution["facts_sha256"]!=sha256_file(run/"artifacts/facts.json"):
        raise ValueError("input facts changed")
    report={"solution_sha256":sha256_file(run/"artifacts/solution.json")
,"tails":[],"online":{},"passed":True}
    for row in solution["tails"]:
        p=list(map(Fraction,row["exact"]["primary"]))
        r=list(map(Fraction,row["exact"]["recovered"]))
        assert sum(p)==Fraction(str(row["tail_m"]))
        assert all(Fraction("4.8")<=v<=Fraction("12.6") for v in p)
        assert all(v==0 or 9<=v<=10 for v in r)
        assert all(x<=y for x,y in zip(r,p))
        assert sum(p)-sum(r)==Fraction(row["exact"]["waste"])
        assert sum((v-Fraction("9.5"))**2 for v in r if
v)==Fraction(row["exact"]["squared_deviation"])
        network=network_optimum(0,0,[],95,90,100,fixed_end=round(row["ta
il_m"]*10))
        assert abs(network["loss_tick"]/10-row["waste_m"])<1e-8
        report["tails"].append({"tail_m":row["tail_m"],"waste_m":network
["loss_tick"]/10,
                                "loss_agreement":True,"network":network,
                                "secondary_scope":"rational allocation
checked algebraically; network secondary cost is grid-restricted, not
the continuous Q1 objective"})
    for name,record in solution["online"].items():
        checked=verify_trace(record,facts["event_times_min"])
        references=[]
        case=record["case"]

```

```

        target, lower, upper=[round(case[k]*10) for k in
("target_m", "lower_m", "upper_m")]
        for row in record["events"]:
            p=row["new_plan"]
            network=network_optimum(p["start"],p["notification"],p["defe
cts"],target,lower,upper)
            assert network["loss_tick"]==p["score"][0]
            assert network["deviation_tick2"]==p["score"][1]

references.append({"event":row["event"],"passed":True,**network})
        bound=continuous_lower_bound(facts["event_times_min"],case["lowe
r_m"],case["upper_m"])
        bound["attained"]=Fraction(str(record["total_waste_m"]))==Fracti
on(bound["exact_waste"])
        report["online"][name]={"trace":checked,"network_comparisons":re
ferences,"continuous_lower_bound":bound}
        print(name,"LP matches all 9 event objectives; continuous waste
bound",bound["waste_m"],"attained",bound["attained"],flush=True)
        write_json(run/"artifacts/independent.json",report)
        return report

if __name__=="__main__":
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument("run",type=Path)
    verify(parser.parse_args()).run()

```